

C 程序综合实训项目：校园实验田环境监测系统

1、项目目标

开发一个完整的校园实验田环境监测系统，使用纯 C 语言实现，所有数据存储在内存中。系统能够管理多个实验田块，记录温度湿度传感器数据，并具备数据查询、分析和预警功能。

2、技术要求

编程语言：标准 C 语言（C99 标准）

开发环境：任选

禁止使用：文件 I/O 操作、第三方库 qsort 等现成排序函数

必须使用：动态内存分配、结构体、指针、链表

3、核心训练点

结构体与指针的综合运用、动态内存管理（malloc/free）、链表和动态数组的实现、字符串处理和时间管理、模块化编程和调试技巧

4、数据结构定义（必须按此实现）

// 实验田块信息（使用动态数组管理）

```
typedef struct Field {  
    int id;                // 田块 ID，从 1 开始自动分配  
    char name[30];         // 田块名称，如"东区实验田 1 号"  
    char manager[20];      // 负责人姓名  
    int sensor_count;      // 该田块已安装的传感器数量  
} Field;
```

// 传感器数据记录（使用链表管理）

```
typedef struct SensorRecord {  
    int field_id;          // 对应的田块 ID  
    char timestamp[20];    // 记录时间，格式"2025-04-10 14:30"  
    float temperature;     // 温度值，单位℃  
    float humidity;        // 湿度值，单位%  
    int is_abnormal;       // 是否异常：0-正常，1-异常  
    struct SensorRecord *next; // 指向下一条记录  
} SensorRecord;
```

// 系统告警信息（使用链表管理）

```
typedef struct Alert {  
    int field_id;          // 发生告警的田块 ID  
    char alert_time[20];   // 告警时间  
    char alert_type[30];   // 告警类型，如"高温告警"  
    char description[100]; // 告警详细描述  
    int severity;          // 严重程度：1-轻微，2-中等，3-严重  
    struct Alert *next;    // 指向下一条告警  
} Alert;
```

```
// 监测系统主结构
typedef struct MonitorSystem {
    Field **fields;           // 田块指针数组（二重指针）
    int field_count;          // 当前田块数量
    int field_capacity;       // 数组容量，初始为 10
    SensorRecord *record_head; // 传感器记录链表头指针
    Alert *alert_head;        // 告警信息链表头指针
    int next_field_id;        // 下一个可用的田块 ID，用于保证每个新创建的田块都有一个唯一且连续的 ID。
} MonitorSystem;
```

重要说明：Field **fields 是一个二重指针，指向一个指针数组，每个 SensorRecord 节点通过链表连接，每个 Alert 节点通过链表连接，所有字符串使用固定大小的字符数组。

5、必须实现的功能函数清单（共 23 个）

(1)、系统管理函数（4 个）

函数原型	功能说明	返回值/输出
MonitorSystem* create_monitor_system(void)	创建并初始化监测系统	成功：指针，失败：NULL
void destroy_monitor_system(MonitorSystem *sys)	销毁系统, 释放所有内存	无
void display_main_menu(void)	显示系统主菜单	输出菜单到屏幕
void expand_field_array(MonitorSystem *sys)	动态扩容田块数组	容量扩大为原来 2 倍

(2)、实验田管理函数（6 个）

函数原型	功能说明	参数说明
int add_field(MonitorSystem *sys, const char *name, const char *manager)	添加新田块	name: 田块名称, manager:负责人
void display_all_fields(MonitorSystem *sys)	显示所有田块	按表格格式输出
Field* find_field_by_id(MonitorSystem *sys, int field_id)	按 ID 查找田块	field_id:要查找的 ID
int delete_field(MonitorSystem *sys, int field_id)	删除指定田块	同时删除相关记录和告警
void search_fields_by_name(MonitorSystem *sys, const char *keyword)	按名称搜索田块	keyword:搜索关键词
void search_fields_by_manager(MonitorSystem *sys, const char *manager_name)	按负责人搜索	manager_name: 负责人姓名

(3)、传感器数据管理函数（6 个）

函数原型	功能说明	异常判断标准
int add_sensor_record(MonitorSystem *sys, int field_id, float temperature, float	添加传感器记录	温度<10℃或>35℃为异常, 湿度 <40%

humidity)		或>85%为异常
void display_latest_records(MonitorSystem *sys, int field_id, int count)	显示最新记录	count:显示条数
void display_field_all_records(MonitorSystem *sys, int field_id)	显示所有记录	按时间排列(距离现在最近, 显示在最前面)
void delete_old_records(MonitorSystem *sys, int days)	删除过期记录	days:保留最近几天的数据
int get_record_count(MonitorSystem *sys, int field_id)	获取记录数量	返回该田块的总记录数
float calculate_avg_temperature(MonitorSystem *sys, int field_id)	计算平均温度	返回平均温度值

(4)、数据分析函数 (3 个)

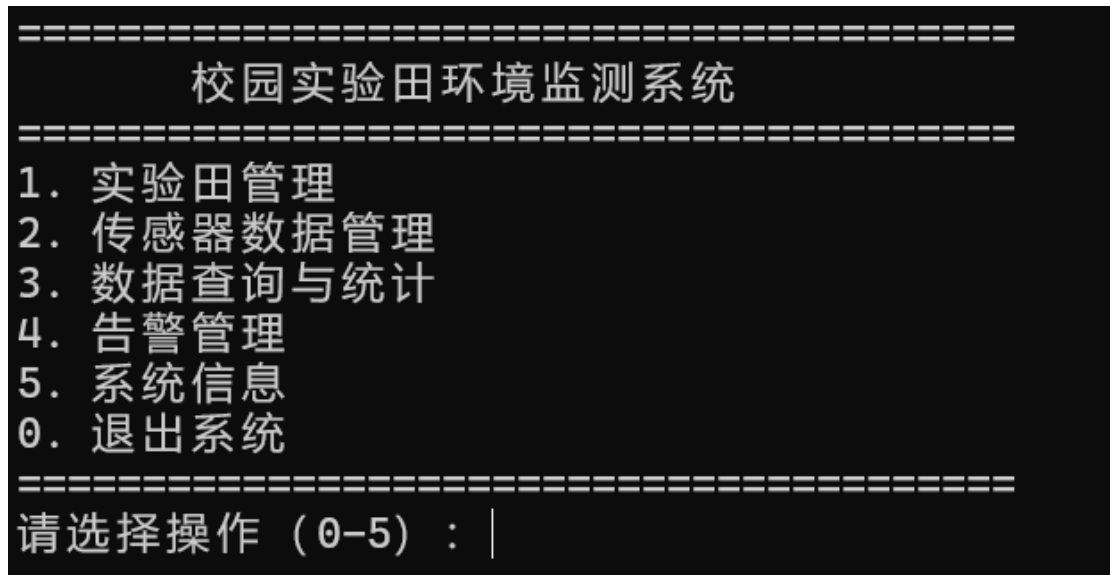
函数原型	功能说明	输出要求
void find_abnormal_records(MonitorSystem *sys, int field_id)	查找异常记录	列出所有异常记录, 显示原因(温度过低、温度过高、湿度过低、湿度过高)
void find_extreme_values(MonitorSystem *sys, int field_id)	查找极值	输出最高/最低温度和湿度
void analyze_trend(MonitorSystem *sys, int field_id)	分析变化趋势	判断上升/下降/稳定趋势, 比如可以分析第一条和最后一条温度, 是上升还是下降, 湿度上升还是下降, 如果小于 2 条数据, 则提示无法分析

(5)、告警管理函数 (4 个)

函数原型	功能说明	告警规则
void check_and_generate_alerts(MonitorSystem *sys, int field_id)	检查并生成告警	温度>35℃:高温告警(2 级); 温度<10℃:低温告警(2 级); 湿度<40%:低湿告警(1 级); 湿度>85%:高湿告警(1 级); 连续 3 条异常:持续异常告警(3 级)
void display_all_alerts(MonitorSystem *sys)	显示所有告警	按严重程度降序排列
int resolve_alert(MonitorSystem *sys, int field_id, const char *alert_type)	处理告警	从链表中删除对应告警
int get_alert_count(MonitorSystem *sys, int field_id)	获取告警数量	返回该田块的告警总数

6、功能要求

(1)、 部分界面 demo



--- 实验田管理 ---

1. 添加新田块
2. 显示所有田块
3. 查找田块 (按ID)
4. 搜索田块 (按名称)
5. 搜索田块 (按负责人)
6. 删除田块
0. 返回主菜单

请选择: 2

=====

所有实验田块 (共5个)

=====

ID	名称	负责人	传感器数
1	东区实验田1号	张三	3
2	西区实验田2号	李四	3
3	南区实验田3号	王五	3
4	北区实验田4号	赵六	3
5	中区实验田5号	钱七	3

=====

--- 实验田管理 ---

1. 添加新田块
2. 显示所有田块
3. 查找田块 (按ID)
4. 搜索田块 (按名称)
5. 搜索田块 (按负责人)
6. 删除田块
0. 返回主菜单

请选择: 6

请输入要删除的田块ID: 2

正在删除田块: 西区实验田2号 (ID: 2)

田块删除成功!

--- 实验田管理 ---

1. 添加新田块
2. 显示所有田块
3. 查找田块 (按ID)
4. 搜索田块 (按名称)
5. 搜索田块 (按负责人)
6. 删除田块
0. 返回主菜单

请选择: |

--- 实验田管理 ---

1. 添加新田块
2. 显示所有田块
3. 查找田块 (按ID)
4. 搜索田块 (按名称)
5. 搜索田块 (按负责人)
6. 删除田块
0. 返回主菜单

请选择: 0

返回主菜单...

=====

校园实验田环境监测系统

=====

1. 实验田管理
2. 传感器数据管理
3. 数据查询与统计
4. 告警管理
5. 系统信息
0. 退出系统

=====

请选择操作 (0-5): 3

--- 数据查询与统计 ---

1. 查找异常记录
2. 查找极值
3. 分析趋势
0. 返回主菜单

请选择: 3

请输入田块ID: 3

田块: 南区实验田3号 (ID: 3) 的趋势分析

=====

基于最近 3 条记录分析:

【温度趋势】基本稳定 (波动0.7°C)

【湿度趋势】基本稳定 (波动2.0%)

--- 数据查询与统计 ---

1. 查找异常记录
2. 查找极值
3. 分析趋势
0. 返回主菜单

请选择: |

```
=====
                        校园实验田环境监测系统
=====
1. 实验田管理
2. 传感器数据管理
3. 数据查询与统计
4. 告警管理
5. 系统信息
0. 退出系统
=====
请选择操作（0-5）： 4

--- 告警管理 ---
1. 检查并生成告警
2. 显示所有告警
3. 处理告警
4. 统计告警数量
0. 返回主菜单
请选择： 2

=====
                        所有系统告警
=====
田块ID   时间                               类型           严重程度   描述
-----
暂无告警信息

--- 告警管理 ---
1. 检查并生成告警
2. 显示所有告警
3. 处理告警
4. 统计告警数量
0. 返回主菜单
请选择： 4
请输入田块ID（输入0查看全部）： 0
系统共有0条告警

--- 告警管理 ---
1. 检查并生成告警
2. 显示所有告警
3. 处理告警
4. 统计告警数量
0. 返回主菜单
请选择： |
```

(2)、输出格式要求

田块显示格式

ID	名称	负责人	传感器数	最新温度	最新湿度	状态
1	东区实验田 1 号	张三	2	24.5℃	68%	正常
2	西区实验田 3 号	李四	1	32.1℃	42%	高温告警

传感器记录格式

时间	温度(℃)	湿度(%)	状态
2025-04-10 14:30	24.5	68	正常
2025-04-10 13:30	32.1	42	异常(高温)
2025-04-10 12:30	18.2	85	异常(高湿)

告警显示格式

田块 ID	时间	类型	严重程度	描述
1	2025-04-10 14:30	高温告警	2	温度达到 36.5℃
2	2025-04-10 13:15	持续异常	3	连续 3 次检测异常

(3)、时间格式要求

所有时间使用 24 小时制

格式: "YYYY-MM-DD HH:MM"

示例: "2025-04-10 14:30"

日期和时间之间有一个空格

7、关键技术要求

(1)、规定二重指针的地方必须使用二重指针

Field **fields; // 必须正确使用

理解: 这是一个指针数组, 每个元素指向一个 Field 结构体

优势: 扩容时只需复制指针, 不需要复制整个结构体

(2)、必须正确管理链表

传感器记录按时间倒序排列 (最新记录在链表头部)

告警信息按严重程度降序排列 (严重告警在前)

每次添加新节点都要正确维护链表结构

(3)、必须自己实现排序算法

// 示例: 冒泡排序实现

```
void sort_alerts_by_severity(Alert *head) {  
    // 必须自己实现, 不能用 qsort  
}
```

(4)、必须处理所有边界情况

空系统操作 (没有田块时)

空链表操作 (没有记录时)

非法输入处理 (如不存在的 ID)

数组越界预防

8、重要提示

(1)、开始前准备

理解所有数据结构, 画出内存布局图

制定开发计划，合理分配 10 天时间
搭建好开发环境，安装必要的调试工具

(2)、开发建议

模块化开发：逐个函数实现，逐个测试
频繁测试：每完成一个功能立即测试
版本控制：每天备份代码
调试技巧：使用 printf 调试，逐步缩小问题范围

(3)、常见问题解决

段错误：检查指针是否为空，数组是否越界
内存泄漏：确保每个 malloc 都有对应的 free
链表问题：检查头指针和 next 指针的更新
排序错误：验证比较函数和交换逻辑

(4)、寻求帮助

先自己思考和搜索解决方案
与同学讨论思路，但不抄袭代码
在答疑时间段提问，描述清晰的问题现象

9、学习目标

完成本项目后，你将能够：
熟练使用结构体和指针、掌握动态内存管理技巧、实现链表和动态数组、设计中等规模的 C 语言程序、调试复杂的内存相关问题，为后续数据结构和算法学习奠定坚实基础！

计算机系